

×

Create Database

Database Name

Collection Name

☐ **Time-Series**
Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

> **Additional preferences** (e.g. Custom collation, Clustered collections)

Cancel

Create Database

- ✓ Inicialmente se crea la base de datos, la cual se nombra entertainment y posteriormente cada una de las colecciones.

×

Import

To collection entertainment.comments

Import file:

Options

☐ Stop on errors

Cancel

Import

- ✓ Luego se insertan los datos para cada una de las colecciones.

localhost:27017 > entertainment

> Open MongoDB shell

+ Create collection

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
comments	-	4.10 kB	14.29 MB	50K	284.00 B	1	4.10 kB
movies	-	4.10 kB	37.62 MB	24K	1.60 kB	1	4.10 kB
sessions	-	4.10 kB	540.00 B	1	540.00 B	1	4.10 kB
theaters	-	4.10 kB	349.83 kB	1.6K	223.00 B	1	4.10 kB
users	-	4.10 kB	29.57 kB	185	159.00 B	1	4.10 kB



Ejercicio 1

➤ Muestra los 2 primeros comentarios que aparecen en la base de datos.

localhost:27017 > entertainment > comments Open MongoDB shell

Documents 50K Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) ⚡

Project { field: 0 }

Sort {date: 1} Max Time MS 60000

Collation { locale: 'simple' } Skip 0 Limit 2

Index Hint { field: -1 } or "indexName"

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

```
{
  "_id": ObjectId("5a9427648b0beebe695e67e"),
  "name": "Mercedes Tyler",
  "email": "mercedes_tyler@fakegmail.com",
  "movie_id": ObjectId("573a1399f29313caabcec3ce"),
  "text": "Optio totam dolores magni. Enim ratione fuga tempora voluptatum est cu...",
  "date": 1970-01-01T01:07:09.000+00:00
}
```

```
{
  "_id": ObjectId("5a9427658b0beebe6969efe"),
  "name": "Jon Snow",
  "email": "kit_harington@gameofthron.es",
  "movie_id": ObjectId("573a13a4f29313caabd117df"),
  "text": "Dolorem animi tempora ullam quas. Iusto nobis reprehenderit aspernatur...",
  "date": 1970-01-01T09:37:49.000+00:00
}
```

- ✓ En este caso se selecciona la colección *comments*, en Sort se ordena por el campo *date* de manera ascendente (1) y en opciones se escribe en *Limit* la cantidad a mostrar que es 2.

➤ ¿Cuántos usuarios tenemos registrados?

localhost:27017 > entertainment > users

Documents 185 Aggregations Schema Indexes 1 Validation

- ✓ Se selecciona la colección *users* y la cantidad que muestra Documents sería el equivalente al número de usuarios registrados (185).

➤ ¿Cuántos cines existen en el estado de California?

➤ ¿Cuántas películas de comedia existen en nuestra base de datos?

The screenshot displays a MongoDB Atlas pipeline editor with two stages:

```
1 [
2   {
3     $unwind:
4       /**
5        * path: Path to the array field.
6        * includeArrayIndex: Optional name for index.
7        * preserveNullAndEmptyArrays: Optional
8        * toggle to unwind null and empty values.
9        */
10    {
11      path: "$genres"
12    }
13  },
14  {
15    $group:
16      /**
17       * _id: The id of the group.
18       * fieldN: The first field name.
19       */
20    {
21      _id: "$genres"
22    }
23  }
24 ]
```

The **PIPELINE OUTPUT PREVIEW** shows a sample of 10 documents with the following `_id` values: "Western", "Short", "War", "Talk-Show", "Comedy", "Fantasy", "News", and "History".

Below the pipeline editor, there are buttons for **SAVE**, **CREATE NEW**, **EXPORT DATA**, and **EXPORT CODE**, along with a **PREVIEW** toggle.

The second stage of the pipeline is a `$match` stage:

```
1 [
2   {
3     $match:
4       /**
5        * query: The query in MQL.
6        */
7     {
8       genres: "Comedy"
9     }
10  },
11  {
12    $group:
13      /**
14       * _id: The id of the group.
15       * fieldN: The first field name.
16       */
17    {
18      _id: null,
19      Num_movies_comedy: {
20        $sum: 1
21      }
22    }
23  }
24 ]
```

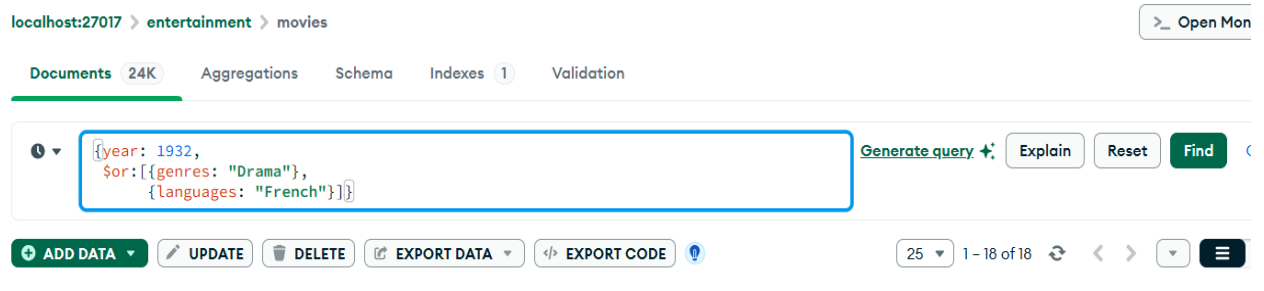
The **PIPELINE OUTPUT PREVIEW** for this stage shows a sample of 1 document:

```
{
  "_id": null,
  "Num_movies_comedy": 7024
}
```

- ✓ Inicialmente se utiliza una agregación para ver los diferentes géneros que existen, así como su escritura correcta, en la cual se utiliza la función *unwind* para descomponer el array y luego se agrupan los diferentes géneros (geners) mediante *group*.
- ✓ Posteriormente se hace otra agregación utilizando *match* para filtrar solo el género Comedy, se agrupa asumiendo el id como nulo y se calcula el total mediante la función *sum*.

Ejercicio 2

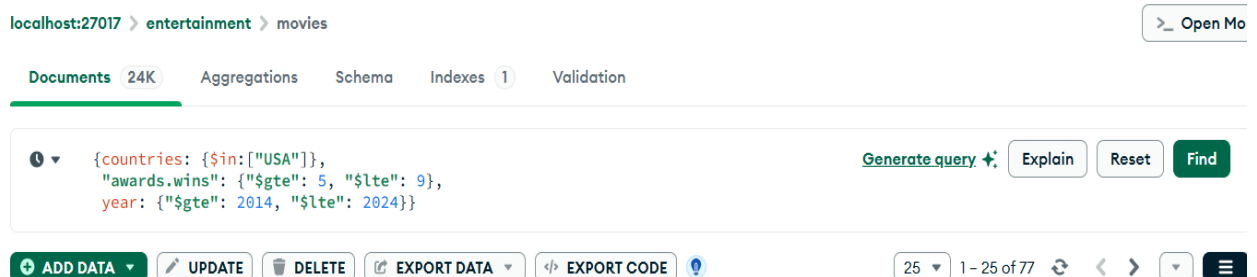
- Muéstrame todos los documentos de las películas producidas en 1932, pero que el género sea drama o estén en francés.



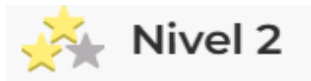
- ✓ Se selecciona la colección *movies*, luego se filtra las condiciones mediante el operador lógico de consulta \$or, aunque los campos genres y languages son arrays no se utiliza el operador \$in ya que se buscará un solo valor en cada caso. Como resultado final se obtienen 18 documentos que cumplen con las condiciones solicitadas.

Ejercicio 3

- Muéstrame todos los documentos de películas estadounidenses que tengan entre 5 y 9 premios que fueron producidas entre 2012 y 2014.



- ✓ Se selecciona la colección *movies*, como el campo countries es un array se utiliza el operador \$in para seleccionar solo los que cumplen la condición, en el campo year para seleccionar solo entre los dos años, se utilizan los operadores \$gte (>=) y \$lte (<=). Además, para los premios, se asumen los ganados por lo cual tenemos en cuenta wins. El resultado serían 25 documentos.



Ejercicio 1

- Cuenta cuántos comentarios escribe un usuario que utiliza "GAMEOFTHRON.ES" como dominio de correo electrónico.

localhost:27017 > entertainment > comments

Documents 50K Aggregations Schema Indexes 1 Validation

Smatch Scount Generate aggregations

Untitled - modified SAVE + CREATE NEW EXPORT DATA EXPORT CODE PREVIEW

```
1 [
2   {
3     $match:
4       /**
5        * query: The query in MQL.
6        */
7     {
8       email: {
9         $regex: "GAMEOFTHRON.ES",
10        $options: "i"
11      }
12    },
13  },
14  {
15    $count:
16      /**
17       * Provide the field name for the count.
18       */
19    "num_comments"
20  }
21 ]
```

PIPELINE OUTPUT PREVIEW
Sample of 1 document

num_comments : 22841

- ✓ Se selecciona la colección *comments*, en este caso se asume que la pregunta está enfocada al total de comentarios que utilizan todos los usuarios del dominio, por lo cual aplicamos una agregación filtrando por el dominio mediante **\$match** y **\$options** para que seleccione indistintamente si está en mayúscula o minúscula el nombre del mismo. Luego se cuenta el total de comentarios mediante el operador **\$count** que resultarían 22841.

localhost:27017 > entertainment > comments

Documents 50K Aggregations Schema Indexes 1 Validation

\$match \$group

Generate aggregation + ?

Untitled - modified

SAVE

+ CREATE NEW

EXPORT DATA

EXPORT CODE

PREVIEW

STAGES

```
1 [
2   {
3     $match:
4       /**
5        * query: The query in MQL.
6        */
7     {
8       email: {
9         $regex: "GAMEOFTHRON.ES",
10        $options: "i"
11      }
12    },
13  },
14  {
15    $group:
16      /**
17       * _id: The id of the group.
18       * fieldN: The first field name.
19       */
20    {
21      _id: "$email",
22      Num_comments: {
23        $sum: 1
24      }
25    }
26  }
27 ]
```

PIPELINE OUTPUT PREVIEW

Sample of 10 documents

_id: "hannah_murray@gameofthron.es"
Num_comments : 265

_id: "peter_dinklage@gameofthron.es"
Num_comments : 266

_id: "brian_fortune@gameofthron.es"
Num_comments : 260

_id: "gemma_whelan@gameofthron.es"
Num_comments : 259

_id: "rory_mccann@gameofthron.es"
Num_comments : 280

_id: "michelle_fairley@gameofthron.es" Ac
Num_comments : 259 Go

- ✓ En este caso se asume que la pregunta está enfocada al total de comentarios por cada usuario, por lo cual se selecciona la colección *comments*, aplicamos una agregación filtrando por el dominio mediante **\$match** y **\$options** para que seleccione indistintamente si está en mayúscula o minúscula el nombre del mismo. Luego se agrupa por cada usuario seleccionando el *email* es un campo único con **\$group** y se obtienen el total de comentarios por usuario.

- ¿Cuántos cines existen en cada código postal situados dentro del estado Washington DC (DC)?

localhost:27017 > entertainment > theaters

Documents 1.6K Aggregations Schema Indexes 1 Validation

\$match **\$group** Generate

Untitled - modified **SAVE** **+ CREATE NEW** **EXPORT DATA** **EXPORT CODE** **PRI**

```
1 [
2   {
3     $match:
4       /**
5        * query: The query in MQL.
6        */
7       {
8         "location.address.state": "DC"
9       }
10  },
11  {
12    $group:
13      /**
14       * _id: The id of the group.
15       * fieldN: The first field name.
16       */
17      {
18        _id: "$location.address.zipcode",
19        Num_theaters: {
20          $sum: 1
21        }
22      }
23  }
24 ]
```

PIPELINE OUTPUT PREVIEW
Sample of 3 documents

_id: "20016"	Num_theaters : 1
_id: "20010"	Num_theaters : 1
_id: "20002"	Num_theaters : 1

- ✓ Se selecciona la colección *theaters*, aplicamos una agregación filtrando con **\$match** los códigos postales correspondientes y luego agrupamos utilizando el campo *zipcode* mediante **\$group** y se suman la cantidad de teatros por cada uno.



Ejercicio 1

- Encuentra todas las películas dirigidas por John Landis con una puntuación IMDb (Internet Movie Database) de entre 7,5 y 8.

localhost:27017 > entertainment > movies

Documents 24K Aggregations Schema Indexes 1 Validation

Smatch Sproject Generate aggregation + Explain Run

Untitled - modified SAVE + CREATE NEW EXPORT DATA EXPORT CODE PREVIEW STAGES TEXT

```
1 [
2   {
3     $match:
4     /**
5      * query: The query in MQL.
6      */
7     {
8       directors: "John Landis",
9       "imdb.rating": {
10        $gte: 7.5,
11        $lte: 8
12      }
13    },
14  },
15  {
16    $project: {
17      title: 1
18    }
19  }
20 ]
```

PIPELINE OUTPUT PREVIEW

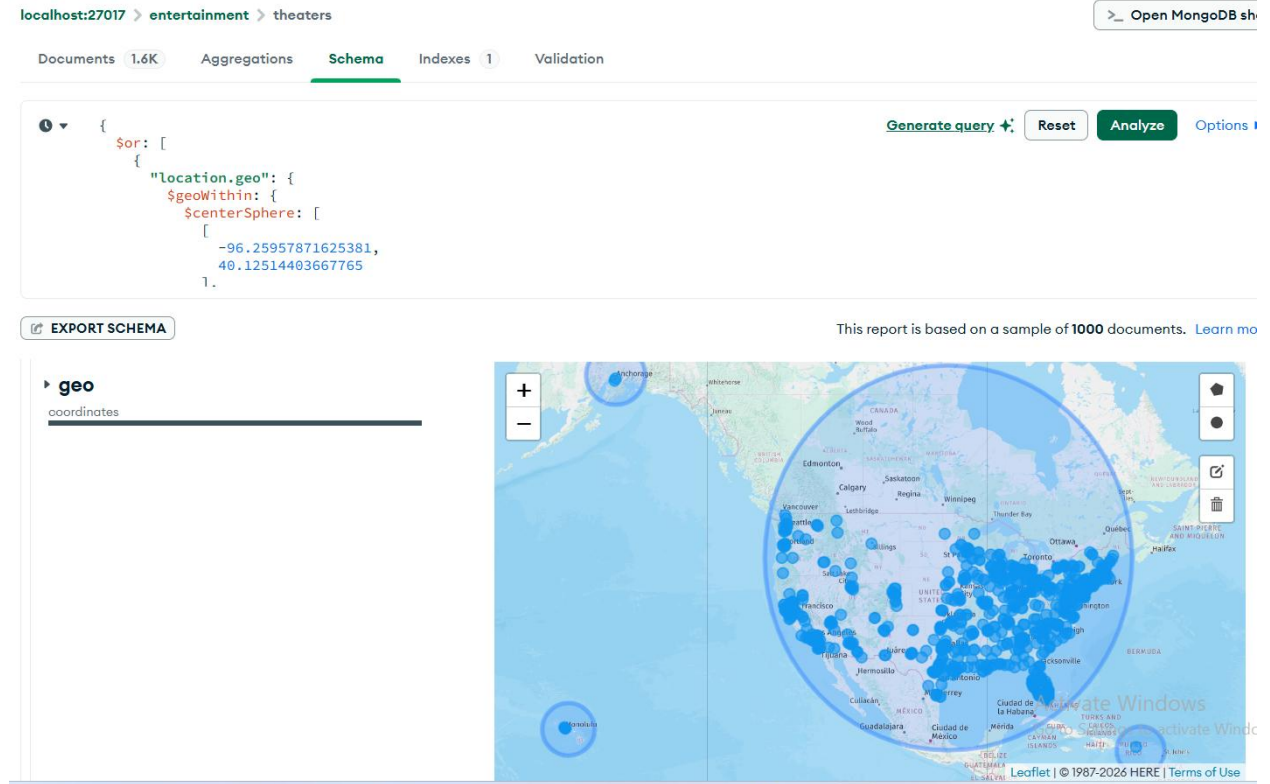
Sample of 4 documents

_id: ObjectId('573a1397f29313caabce6d94')	title: "Animal House"
_id: ObjectId('573a1397f29313caabce76f7')	title: "The Blues Brothers"
_id: ObjectId('573a1397f29313caabce7d06')	title: "An American Werewolf in London"
_id: ObjectId('573a1398f29313caabce8deb')	title: "Trading Places"

- ✓ Se selecciona la colección *movies*, utilizamos una agregación filtrando con **\$match** los filmes con el director y el rating entre los valores correspondientes y finalmente mostramos solo el título mediante el operador **\$project**.

Ejercicio 2

➤ Muestra en un mapa la ubicación de todos los teatros de la base de datos.



- ✓ Es posible mostrar la ubicación ya que la colección tiene un campo que contiene las coordenadas de ubicación, así que en la pestaña Schema vamos al campo coordinates y podemos visualizar la ubicación de los teatros.